La biblioteca: buscar en los documentos y citar

Asesoría del proyecto final — Semana 2

Carlos Orrego

Razonamiento Automático — Prof. Fabián Serrano
Universidad Nacional de Colombia — Sede Manizales

12 de noviembre de 2026

Hoy, en una frase

Al salir de esta clase. . .

. . . el modelo ya no responde de memoria: antes de cada pregunta busca en los documentos del curso, y cita lo que usó.

Es la configuración 2: el mismo modelo de la semana pasada, con libro abierto.

Recorrido

Minutos	Parte
0–10	1. La tarea: ¿cuánto dio la línea base?
10–20	2. Examen a libro abierto
20–40	3. Las piezas: normalizar, fragmentar, BM25
40–50	4. Medir la búsqueda sin gastar el modelo
50–70	5. Taller: configuración 2 y citas
70–80	6. Tarea

Los cuatro jueves

Jueves	Nombre	Construimos
1	El hilo	el modelo y la línea base
2 (hoy)	La biblioteca	buscar en los documentos y citar
3	La flecha roja	verificar con Isabelle y corregir
4	El espejo	evaluar, analizar errores, informe

¿Cuánto dio su línea base?

- Configuración 1 sobre las 58: ¿cuántas `aceptada`, `estilo`, `rechazada`, `trampa`?
- De los diez registros que leyeron: ¿qué errores se repiten?
- ¿Qué tema de formalización eligió cada uno?

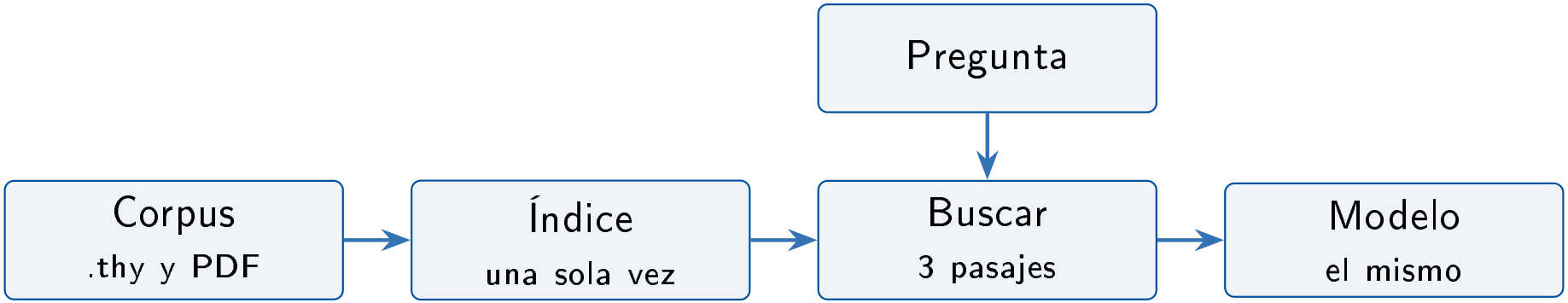
Ese número es contra el que se mide todo lo que hagamos hoy.

El modelo pequeño recuerda poco

- La semana pasada vimos a Gemma inventar `module Main`, `assert` y reglas `and`, `or`.
- No es falta de inteligencia: Isabelle es un rincón pequeño de lo que el modelo leyó.
- Pero el curso tiene 588 lemas resueltos, los dos libros de Alonso y las diapositivas.

¿Y si le damos el libro abierto en la página correcta?

La configuración 2



- Todavía sin Isabelle dentro del asistente: eso es el jueves 3.
- Mismo modelo y misma temperatura que la configuración 1. Si no, la comparación no mide la búsqueda.

Pieza 1: normalizar los símbolos

En el `.thy` dice `\<and>`; en el PDF y en el modelo, $\wedge$. Para la búsqueda deben ser la misma palabra.

```
SIMBOLOS = {
    "and":     ["\\<and>", ""],
    "implies": ["\\<longrightarrow>", "", "-->"],
    "entails": ["\\<Longrightarrow>", "", "==>", "\\<turnstile>"],
    ...
}
```

`normalizar.py`. Se aplica al indexar y al buscar.

Pieza 2: fragmentar por comandos, no por tamaño

- Un fragmento empieza en un comando: `lemma`, `fun`, `definition`, `section`...
- Guarda archivo, línea y nombre: con eso se cita.
- El `text` de antes se pega al lema: ahí dice «Ejercicio 3. Demostrar . . . ».
- Un lema que termina en `oops` no trae demostración: para demostrar no sirve.
- Los PDF, por página.

Resultado: 2 318 fragmentos, 588 lemas con demostración, en 3 segundos.

Pieza 3: BM25

Para cada fragmento, cada palabra de la consulta que aparece en él suma puntos. Suma más si:

- aparece varias veces, con techo;
- es rara en el corpus: `conjE` pesa más que `by`;
- el fragmento es corto.

¿Por qué no *embeddings*?

Las consultas importantes son literales: una regla, una función, una fórmula. BM25 son 30 líneas de Python sin descargar nada, y se explica completo en la sustentación.

Primero medir la búsqueda, después el modelo

- Si la búsqueda no encuentra el pasaje, el modelo no tiene con qué mejorar, y uno culpa al modelo.
- En R1 se puede medir sin modelo: la solución del ejercicio *n* es `ejercicio_n_1` en `R1_Deduccion_natural.thy`.
- `medir_recuperacion.py` cuenta en qué puesto queda.

	primera	entre las 3 primeras
Palabras sueltas	3/58	15/58

¿Por qué tan mal?

$p \longrightarrow q$ y $q \longrightarrow p$ son idénticas

Con palabras sueltas, las dos son {p, q, implies}. En lógica proposicional todas las fórmulas usan las mismas palabras.

El arreglo es una línea: contar también los pares de palabras seguidas.

```
def tokens(texto):
    t = re.findall(r"[a-z_][a-z0-9_']*|[0-9]+", normalizar(texto))
    return t + [a + "+" + b for a, b in zip(t, t[1:])]
```

	primera	entre las 3 primeras
Palabras sueltas	3/58	15/58
Con pares	57/58	58/58
Con pares, sin el número del ejercicio	41/58	56/58

La última fila es la honesta: en el benchmark privado no habrá «Ejercicio 3» que coincida.

Paso 1: traer y construir el índice

```
cd repo_proyecto_final
git pull
cd semana2/taller
python3 indexar.py
python3 buscar.py "conjE"
python3 medir_recuperacion.py
```

Para los PDF: `pip install pypdf` o `brew install poppler`. Sin ellos se indexan solo las teorías, y para hoy basta.

Paso 2: lo que recibe el modelo

```
Fragmentos del corpus del curso que pueden servir...

[F1] (TeoriasIsabelle/R1_Deduccion_natural.thy, linea 55 (ejercicio_1_1))
text \<open> Ejercicio 1. Demostrar ... \<close>
lemma ejercicio_1_1:
  assumes 1: "p \<longrightarrow> q" and 2: "p"
  shows "q"
proof - ...

[F2] ...   [F3] ...

Enunciado:
lemma ejercicio_1: ...
```

3 fragmentos, cada uno cortado en 1 500 caracteres: unos 4 700 en total.

Un error nuevo: el modelo copia el fragmento

- Con libro abierto, el modelo copia: el `text`, la línea `lemma ejercicio_1_1:` y a veces el lema siguiente.
- Para `evaluar.py` eso es trampa: trae `lemma`.
- `limpiar.py`: desde la primera palabra que abre una prueba (`proof`, `by`, `using`...) hasta el siguiente comando. Si no hay ninguna, vacío.

Lo encontramos con un modelo falso que copiaba el primer fragmento. Probar las tuberías con un modelo falso es barato y encuentra errores así.

Paso 3: correr y calificar

```
python3 config2.py 3              # primero 3
python3 config2.py                # las 58
python3 ../../Recursos/evaluar.py ../../Recursos/benchmark_R1.json \
        respuestas_config2.json --paralelo 4 --informe informe_c2.json
```

- Usa el `modelo.py` de la semana 1: el mismo modelo.
- `registros/config2.jsonl` guarda también los pasajes y su puntaje (R5).

Paso 4: preguntas con citas

```
python3 preguntar.py "Que diferencia hay entre rule y erule?"
```

- El modelo cita por etiqueta: [F2].
- El programa traduce la etiqueta en archivo y línea, o documento y página. El modelo no escribe rutas.
- Si usa una etiqueta que no existe: `!! CITA INVENTADA`.

Que la etiqueta exista no garantiza que el fragmento diga eso. Eso se revisa a mano.

Tarea para el jueves 3

1. Índice construido y `medir_recuperacion.py` corrido.
2. Configuración 2 sobre las 58, mismo modelo, calificada. ¿Cuánto subió? ¿Cuántas `estilo`?
3. Preguntas del anexo A con `preguntar.py`, y cada cita revisada a mano.
4. Tres preguntas trampa: premisa falsa, fuera de tema, y una que el corpus no responde.
5. Formalización: cinco enunciados escritos. Indexen sus soluciones y midan si la búsqueda las encuentra.

El jueves 3: la flecha roja

Hoy el modelo lee antes de escribir.

El jueves, Isabelle le devuelve el error
y el modelo corrige.